

Shell编程

1. Shell介绍

1.1 简介

Shell 是一个用 C 语言编写的程序，通过 Shell 用户可以访问操作系统内核服务

Shell类似于DOS下的command和后来的 cmd.exe

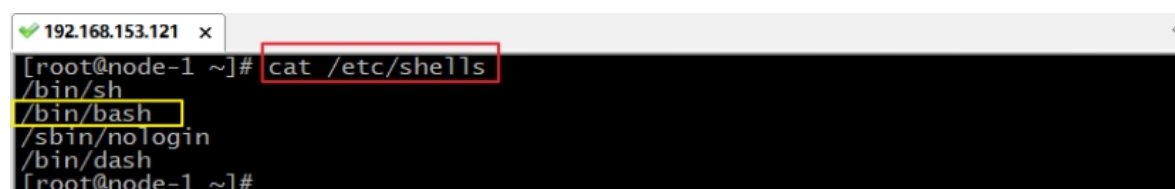
Shell既是一种命令语言，又是一种程序设计语言

Shell script 是一种为shell编写的脚本程序。Shell编程一般指shell脚本编程，不是指开发shell自身

1.2 Shell解释器

Shell编程跟传统的编程语言一样，只要有一个能编写代码的文本编辑器和一个能解释执行的脚本解释器就可以了。

Linux的Shell解释器种类众多，一个系统可以存在多个 shell，可以通过 `cat /etc/shells` 命令查看系统中安装的 shell



```
192.168.153.121 x
[root@node-1 ~]# cat /etc/shells
/bin/sh
/bin/bash
/sbin/nologin
/bin/dash
[root@node-1 ~]#
```

bash 由于易用和免费，在日常工作中被广泛使用。同时， bash 也是大多数 Linux 系统默认的 Shell。

2. 快速入门

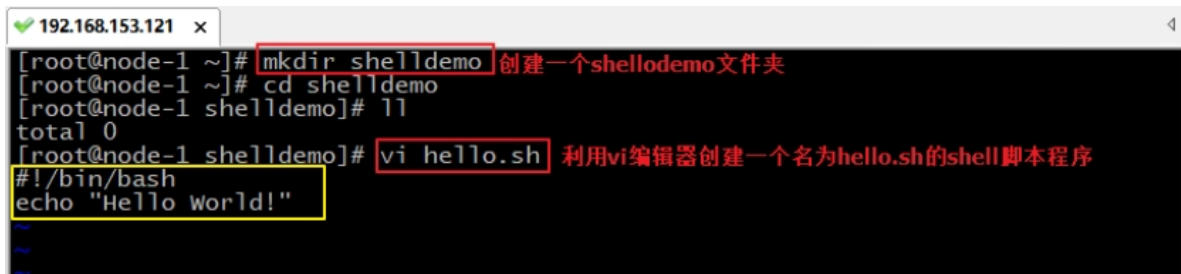
2.1 编写Shell脚本

使用 vi 编辑器新建一个文件hello.sh（扩展名并不影响脚本执行）

```
1 |#!/bin/bash
2 |echo "Hello World !"
```

#! 是一个约定的标记，它告诉系统这个脚本需要什么解释器来执行，即使用哪一种 Shell

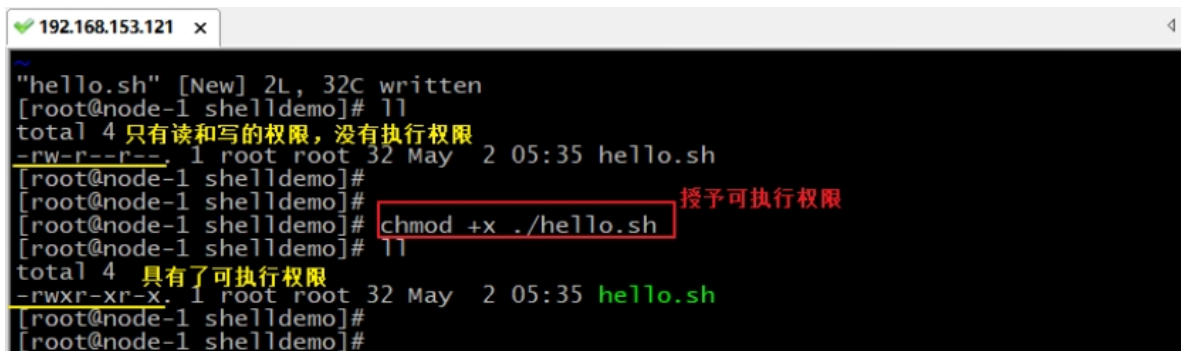
echo命令 用于向窗口输出文本。



```
192.168.153.121 x
[root@node-1 ~]# mkdir shelldemo 创建一个shelldemo文件夹
[root@node-1 ~]# cd shelldemo
[root@node-1 shelldemo]# ll
total 0
[root@node-1 shelldemo]# vi hello.sh 利用vi编辑器创建一个名为hello.sh的shell脚本程序
#!/bin/bash
echo "Hello World!"
```

给shell程序赋予执行权限：

```
1 |chmod +x ./hello.sh # 使脚本具有执行权限
```



```
192.168.153.121 x
"hello.sh" [New] 2L, 32C written
[root@node-1 shelldemo]# ll
total 4 只有读和写的权限，没有执行权限
-rw-r--r--. 1 root root 32 May  2 05:35 hello.sh
[root@node-1 shelldemo]#
[root@node-1 shelldemo]#
[root@node-1 shelldemo]# chmod +x ./hello.sh 授予可执行权限
[root@node-1 shelldemo]# ll
total 4 具有了可执行权限
-rwxr-xr-x. 1 root root 32 May  2 05:35 hello.sh
[root@node-1 shelldemo]#
[root@node-1 shelldemo]#
```

2.2 执行Shell脚本

执行shell程序：

```
1 | ./hello.sh #执行脚本
```

```
192.168.153.121 x
[root@node-1 shelldemo]#
[root@node-1 shelldemo]# hello.sh 直接写 hello.sh，linux 系统会去 PATH 里寻找有没有叫 hello.sh 的
-bash: hello.sh: command not found
[root@node-1 shelldemo]#
[root@node-1 shelldemo]# ./hello.sh 在当前目录找hello.sh并执行 .表示当前目录
Hello World!
[root@node-1 shelldemo]#
[root@node-1 shelldemo]# /root/shelldemo/hello.sh 全路径指定hello.sh并执行
Hello World!
[root@node-1 shelldemo]#
[root@node-1 shelldemo]# sh hello.sh 运行解释器，并把hello.sh作为参数
Hello World!
[root@node-1 shelldemo]# 直接运行解释器，其参数就是 shell 脚本的文件名
[root@node-1 shelldemo]#
```

直接写 hello.sh，linux 系统会去 PATH 里寻找有没有叫hello.sh的。

用 `./hello.sh` 告诉系统说，就在当前目录找

还可以作为解释器参数运行。直接运行解释器，其参数就是 shell 脚本的文件名，如：

```
1 | sh /root/shelldemo/hello.sh
```

在使用解释器直接执行shell程序这种方式来运行脚本，不需要考虑脚本程序的执行权限了

```
192.168.153.121 x
[root@node-1 shelldemo]# ll
总用量 1
-rw-r--r--. 1 root root 32 5月 6 05:46 hello.sh
[root@node-1 shelldemo]# ./hello.sh
-bash: ./hello.sh: 权限不够
[root@node-1 shelldemo]# sh hello.sh
Hello World!
[root@node-1 shelldemo]#
```

小结：

使用vi编辑器，创建shell程序文件。通常使用.sh作为shell程序后缀名。

shell程序的基本格式：

- 1、指定解释器信息。默认： `/bin/bash`
- 2、书写shell程序代码
- 3、保存shell程序代码

4、执行shell程序 提前：给shell程序授予可执行权限

第一种： `./xxx.sh` 在当前目录中执行shell程序

第二种： `/xx/xxx.sh` 书写全路径的shell程序

第三种： `sh /xx/xxx.sh` 把shell程序作用/bin/sh解释器的参数，通过运行解释器来执行shell

一个程序的基本组成：

1. 变量
2. 数据类型
3. 运算符号
4. 流程控制语句（默认：程序是按照从上向下依次执行）
5. 数组
6. 函数(另一个名字：方法)

3. Shell程序：变量

3.1 语法格式

变量的语法： `变量名=值`

```
1 | # 变量的名字是由写程序的人自主定义的。 通常在shell定义格式为： 一  
   | 个小写字母单词、 多个单词组成时每个单词之间使用"_"分隔  
2 | your_name="bigdata.com"      # 通常开发中命名惯例： 见名知其意。  
   | 例： name、 gender、 student_name
```

注意： **等号两边不能有空格**，同时变量名的命名须遵循如下规则：

- 首个字符必须为字母（a-z，A-Z）
- 中间不能有空格，可以使用下划线（_）
- 不能使用标点符号

- 不能使用 `bash` 里的关键字（可用 `help` 命令查看保留关键字）

3.2 变量使用

使用一个定义过的变量，只要在变量名前面加 `$` 即可。

```
1 | your_name="bigdata.com"
2 | echo $your_name
3 | echo ${your_name}
```

- 花括号是可选的，加不加都行，加花括号是为了帮助解释器识别变量的边界。

```
1 | #!/bin/bash
2 | name="bigdata"
3 | echo $name
4 | name="hadoop"
5 | echo @$name
```

- 已定义的变量，可以被重新定义。

使用 `readonly` 命令可以将变量定义为只读变量，只读变量的值不能被改变。

语法格式：

```
1 | readonly 变量名=值
```

使用 `unset` 命令可以删除变量。不能删除只读变量。

```
1 | # 定义只读变量
2 | readonly variable_name
3 | unset variable_name # 不能删除只读变量
```

示例：

```
1 | # 定义只读变量
2 | readonly name="bigdata"
3 | # 只读变量不能修改
```

```
4 name="linux"
5 echo $name
6
7 # 不能删除只读变量
8 unset name
```

小结：

变量的定义： `变量名=初始值` 等号两边不能有空格

变量的使用： `$变量名` 或 `${变量名}`

修改变量中的值： `变量名=新的值` 针对普通变量

只读变量： `readonly 变量名=初始值` 只读变量在初始化后不能修改初始值，只读变量不能被删除

删除变量（只能删除普通变量）： `unset 变量名`

3.3 变量类型

变量的类型可以分为：局部变量、全局变量

局部变量：局部变量在脚本或命令中定义，仅在当前 shell 实例中有效，其他 shell 启动的程序不能访问局部变量。

```
1 # 在当前实例 下
2 name="hadoop"
3 echo $name      # 可以正常输出
```

```
1 # 新建一个连接实例
2 echo $name
```

全局变量(环境变量): 所有的程序, 包括shell启动的程序, 都能访问环境变量, 有些程序需要环境变量来保证其正常运行。可以用过 `set` 命令查看当前环境变量。

```
192.168.153.121 [root@hadoop-node1 shelldemo]# set
BASH=/bin/bash
BASHOPTS=checkwinsize:cmdhist:expand_aliases:extquote:force_ignores:hostcomplete:interactive_comment
s:login_shell:progcomp:promptvars:sourcpath
BASH_ALIASES=()
BASH_ARGC=()
BASH_ARGV=()
BASH_CMDS=()
BASH_LINENO=()
BASH_SOURCE=()
BASH_VERSINFO=( [0]="4" [1]="1" [2]="2" [3]="1" [4]="release" [5]="x86_64-redhat-linux-gnu")
BASH_VERSION='4.1.2(1)-release'
```

小结:

变量的定义:

```
1 | 变量名=值      # 等号两边没有空格
```

变量的使用:

```
1 | val=100
2 | echo ${val}      # 使用${变量名}访问变量
3 |
4 | val="hello"
5 | echo $val        # 变量可以重复赋值
```

变量的类型: 全局变量(环境变量)、局部变量(本地变量)

4. 字符串

字符串是shell编程中最常用最有用的数据类型(除了数字和字符串, 也没啥其它类型好用了), 字符串可以用单引号, 也可以用双引号, 也可以不用引号。

4.1 单引号

示例:

```
|
```

```
1 skill='linux'
2
3 str='I am goot at $skill'
4
5 echo $str
```

- 输出结果: I am goot at \$skill

单引号字符串的限制:

- 单引号里的任何字符都会原样输出, 单引号字符串中的变量是无效的
- 单引号字符串中不能出现单独一个的单引号 (对单引号使用转义符后也不行), 但可成对出现, 作为字符串拼接使用

4.2 双引号

示例:

```
1 skill='linux'
2
3 str="I am goot at $skill"
4
5 echo $str
```

- 输出结果: I am goot at linux

双引号的优点:

- 双引号里可以有变量
- 双引号里可以出现转义字符

4.3 获取字符串长度

示例:

```
1 skill='hadoop'
```



```
2  
3 echo ${skill}    # 输出结果: hadoop  
4  
5 echo ${#skill}   # 输出结果: 6
```

4.4 提取子字符串

```
1 substring(2)      # 从2开始截取到最后  
2 substring(2,3)    # 从2开始截取3个
```

以下实例从字符串第 2 个字符开始截取 4 个字符：

```
1 skill=hadoop  
2 str="I am goot at $skill"  
3  
4 echo ${str:2}      # 输出结果为: am goot at hadoop  
5  
6 echo ${str:2:2}    # 输出结果为: am
```

注意：当字符串中有空格时，空格也算一个字符存在（字符串是从0开始计算）

4.5 查找子字符串

查找子字符串的位置：

```
1 str="I am goot at hadoop"  
2  
3 echo `expr index "$str" am` # 输出是: 3
```

注意： 以上脚本中`是反引号(Esc下面的)，而不是单引号'，不要看错了哦。



小结：

获取字符串长度：

```
1 | ${#字符串}
```

提取子字符串：

```
1 | ${字符串:索引}    ##索引从0开始计算
2 |
3 | ${字符串:索引:长度}
```

查找子字符在字符串中的位置： 从1开始计算位置

```
1 | `expr index 字符串 子字符串`
```

```
1 | ### 回顾上午内容
2 |
3 | # 创建shell脚本程序： 打印hello
4 | vim shellscript.sh
5 |
6 | # 编写程序
7 | #!/bin/bash #告知linux使用哪个解释器
8 | echo "Hello"
9 |
10 | # 执行shell程序
11 | chmod u+x shellscript.sh #修改文件的执行权限
12 | sh /目录/shellscript.sh
```

```

13
14 # 在shell脚本程序中定义普通变量
15 变量名="数据值"      # 等号两边不能有空格
16 echo $变量名
17 echo ${变量名}
18
19 # 定义一个只读变量 (定义后: 不能修改、不能使用unset删除)
20 readonly 变量名=数据值
21
22 # 删除普通变量
23 unset 变量名
24
25 #单引号字符串: 原样输出
26 #双引号字符串: 可以识别字符串的变量、可以使用转义字符
27 name='字符串'
28 str='这是$name'    # $name不能引用变量
29 str="这是一个$name" #可以引用name变量
30
31 #获取字符串的长度
32 name="Hadoop&Spark"
33 len=${#name}
34 echo "字符串$name的长度为: ${len}"

```

5. Shell程序：参数传递

在执行Shell程序脚本时，是可以向shell程序传递参数。

```

1 | sh shellscript.sh # 启动运行解释器, 并把"shellscript.sh"作为
   | 参数传递过去

```

5.1 参数传递方式

传递参数的方式:

```

1 | ./shell程序 [空格] 参数1 [空格] 参数2 ...

```

shell程序脚本内获取参数的格式为： `$n`

n 代表一个数字， 1 为执行脚本的第一个参数， 2 为执行脚本的第二个参数，以此类推.....

`$0` 表示当前脚本名称

```
[root@hadoop-node1 shelldemo]# vi hello.sh
#!/bin/bash
echo $0
echo $1
echo $2
```

```
[root@hadoop-node1 shelldemo]# ./hello.sh A 100
./hello.sh $0
A
100
```

5.2 特殊字符

shell程序中的特殊字符：

<code>\$#</code>	传递到脚本的参数个数
<code>\$*</code>	以一个单字符串显示所有向脚本传递的参数。
<code>\$\$</code>	脚本运行的当前进程 ID 号
<code>!</code>	后台运行的最后一个进程的 ID 号
<code>@</code>	与 <code>\$*</code> 相同，但是使用时加引号，并在引号中返回每个参数。
<code>?</code>	显示最后命令的退出状态。0 表示没有错误，其他任何值表明有错误。

示例：

```
1  #!/bin/bash
2
3  echo "第一个参数为: $1";
4  echo "参数个数为: $#";
5  echo "传递的参数作为一个字符串显示: $*";
6
7  执行脚本: ./demo1.sh 1 2 3
```

```
1 node-1 x
[root@node-1 shelldemo]# vi demo1.sh

#!/bin/bash
echo "第一个参数为: $1";
echo "参数的个数为: $#";
echo "传递的参数作为一个字符串显示: $*";
```

```
1 node-1 x
[root@node-1 shelldemo]# ll
总用量 8
-rw-r--r--. 1 root root 132 5月  4 06:23 demo1.sh
-rwxr-xr-x. 1 root root  88 5月  4 04:28 hello.sh
[root@node-1 shelldemo]# chmod +x demo1.sh
[root@node-1 shelldemo]# ll
总用量 8
-rwxr-xr-x. 1 root root 132 5月  4 06:23 demo1.sh
-rwxr-xr-x. 1 root root  88 5月  4 04:28 hello.sh
[root@node-1 shelldemo]# ./demo1.sh 1 2 A 1 2 A 属于参数。传递给demo1.sh脚本程序
第一个参数为: 1
参数的个数为: 3
传递的参数作为一个字符串显示: 1 2 A
[root@node-1 shelldemo]#
```

5.2 \$*和\$@的区别

相同点：都表示传递给脚本的所有参数。

不同点：

- 不被" "包含时：\$*和\$@都以\$1 \$2... \$n 的形式组成参数列表
- 被" "包含时：
 - "\$*" 会将所有的参数作为一个整体，以"\$1 \$2 ... \$n" 的形式组成一个整串
 - "\$@" 会将各个参数分开，以"\$1" "\$2" ... "\$n" 的形式组成一个参数列表

```
1 node-1 x
[root@node-1 shelldemo]# vi demo1.sh
#!/bin/bash
echo "$@"
echo "$*"

```

```
1 node-1 x
[root@node-1 shelldemo]# sh demo1.sh 1 2 3
1 2 3
1 2 3
[root@node-1 shelldemo]#
[root@node-1 shelldemo]#
```

“\$*” 输出: “1 2 3”

“\$@” 输出: 1 2 3 当作一个完整的参数

小结:

运行shell程序时传递参数: `./shell程序 参数a1 参数2 参数n`

shell程序中接收参数的方式: `$n` n代表一个数字, \$1表示第一个参数

```
1 echo $1 # 接收第一个参数
2
3 $# #参数的个数
4
5 $* #参数列表 “$*”作为一个完整的字符串
6
7 @$ #参数列表 “@$”作为一个参数列表
```

6. Shell程序: 运算符

6.1 运算符的基本使用

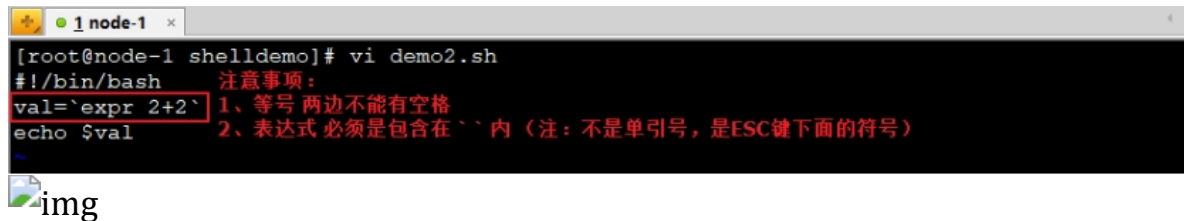
Shell和其他编程语言一样, 支持包括: 算术、关系、逻辑、字符串等运算符。

原生 `/bin/bash` 不支持简单的数学运算，但是可以通过其他命令来实现，例如：`expr`

`expr` 是一款表达式计算工具，使用它能完成表达式的求值操作。

例如，两个数相加：

```
1 | val=`expr 2+2`  
2 | echo $val
```



```
[root@node-1 shelldemo]# vi demo2.sh  
#!/bin/bash  
val=`expr 2+2`  
echo $val  
~
```

img



```
[root@node-1 shelldemo]# vi demo2.sh  
#!/bin/bash  
val=`expr 2 + 2`  
echo $val  
~
```



```
[root@node-1 shelldemo]# sh demo2.sh  
4  
[root@node-1 shelldemo]#
```

注意：

1、运算数和运算符之间要有空格。例如：`2+2` 是不能运算的，必须写成

`2 + 2`

2、完整的表达式要被```符号包含，注意不是单引号，在 `Esc` 键下边

此外，还可以通过`()`、`$[]` 进行算术运算

案例1: `(( ))` 进行算术运算

```
1 | #!/bin/bash  
2 |  
3 | count=1  
4 | ((count++)) ##当仅只是进行+1运算时, 可以直接使用: ((...++))
```

```
5 | echo $count
```

```
1 node-1 x
[root@node-1 shelldemo]# vi demo2.sh
#!/bin/bash
count=1
((count++))
echo $count
```

```
1 node-1 x
[root@node-1 shelldemo]# sh demo2.sh
2
[root@node-1 shelldemo]#
```

案例2: `$(( ))`

```
1 | #!/bin/bash
2 |
3 | val=$((1+1))
4 | echo $val
```

```
1 node-1 x
[root@node-1 shelldemo]# vi demo2.sh
#!/bin/bash
val=$((1+1))
echo $val
```

```
1 node-1 x
[root@node-1 shelldemo]# sh demo2.sh
2
[root@node-1 shelldemo]#
```

案例3: `$[ ]`

```
1 | #!/bin/bash
2 |
3 | val=$[1+2]
4 | echo $val
```



```
1 node-1 x
[root@node-1 shelldemo]# vi demo2.sh
#!/bin/bash
val=$((1+2))
echo $val
```

```
1 node-1 x
[root@node-1 shelldemo]# sh demo2.sh
3
[root@node-1 shelldemo]#
```

6.2 关系运算符

注意：关系运算符只支持数字，不支持字符串，除非字符串的值是数字。

1 | 数字的关系运算符： > >= < <= != ==

运算符	含义
-eq	检测两个数是否相等，相等返回 true
-ne	检测两个数是否不相等，不相等返回 true
-lt	小于 应用于：整型比较。例：10 lt 5
-gt	检测左边的数是否大于右边的，如果是，则返回 true
-le	小于或等于 应用于：整型比较
-ge	大于或等于 应用于：整型比较

6.3 逻辑运算符

运算符	含义
-a	双方都成立（and） 表达式1 -a 表达式2
-o	单方成立（or） 表达式1 -o 表达式2

1 | # && 表示并且 (and)

```
2 echo "$[10>5 && 10%2==0]"
3
4 # || 表示或者 (or)
5 echo "$((10>100 || 10%2==0))"
6
```

```
1 # 直接通过echo方式使用逻辑运算，运算符： && 、 ||
2
3 # 在if语句中使用逻辑运算，运算符： -a 、 -o
4 if [10>5 -a 10>0]
```

6.4 字符串运算符

字符串

运算符	含义
-n STRING	字符串长度不为零（非空字符串）
-z STRING	字符串长度为0（空字符串）
=	判断两个字符串是否一样
!=	判断两个字符串是否不一样

```
1 if ['bigdata'='data']
```

6.5 文件

文件测试运算符

运算符	含义	示例
-f	存在且是普通文件	[-f 文件路径]
-d	存在且是目录	[-d 目录路径]
-s	文件不为空	[-s 文件路径]
-e	文件存在	[-e 文件路径]
-r	文件存在并且可读	[-r 文件路径]
-w	文件存在并且可写	[-w 文件路径]
-x	文件存在并且可执行	[-x 文件路径]

7. 流程控制

任何程序都有默认的执行流程，通常是 **从上向下** 逐行依次执行。

```
1  #!/bin/bash      #先执行的第1行
2  num=100          #第2行
3  ((num++))        #第3行
4  echo $num        #第4行
5  .....
```

当希望对程序的默认执行流程进行控制，需要学习：流程控制

- 选择：有选择性的执行某行或某段程序
- 重复：一直重复性的执行某行或某段程序，至到执行结束(条件控制循环执行的次数)

Shell提供了丰富的语句判断方式，包括数字，字符串和文件。

7.1 if...else

格式1：单支

```
1  if [ 条件 ]; then
```

```
2 | 命令...
3 | fi
```

- 执行机制：判断一次，仅有一个结果
 - 条件成立（true）：执行命令
 - 条件失败（false）：没有任何执行

```
1 | #!/bin/bash
2 | num1=$1 # 把参数1中的数据，赋值给变量num1
3 | num2=$2
4 | if [ $num1 -gt $num2 ]; then
5 |     echo "$num1 大于 $num2"
6 | fi
7 |
8 | ## 执行shell程序
9 | ./ifdemo.sh 10 5
```

格式2：双支

```
1 | if [ 条件 ]; then
2 |     命令1...
3 | else
4 |     命令2...
5 | fi
```

- 执行机制：判断一次条件，有两个不同结果
 - 条件成立（true）：执行 then 后面的代码（命令1）
 - 条件失败（false）：执行 else 后面的代码（命令2）

```
1 | #!/bin/bash
2 | num1=$1
3 | num2=$2
4 | if [ $num1 -gt $num2 ]; then
5 |     echo "$num1 大于 $num2"
6 | else
7 |     echo "$num1 小于 $num2"
8 | fi
```

格式3：多支

```

1  if [ 条件1 ]; then
2      命令1...
3  elif [ 条件2 ]; then
4      命令2
5      .....elif
6  else
7      默认命令...
8  fi

```

- 执行机制：有多个判断条件，每个判断条件对应一个结果；如果所有的判断条件都不成立，则执行else后面的默认结果
 - 当第1个判断条件就成立了，会执行命令1。后续其他的判断条件都不会再执行了

```

1  #!/bin/bash
2  score=$1
3  if [ $score -ge 90 ]; then
4      echo "优秀"
5  elif [ $score -ge 80 ]; then
6      echo "良好"
7  elif [ $score -ge 60 ]; then
8      echo "及格"
9  else
10     echo "不及格"
11 fi

```

案例1：条件判断

```

root@node-1:~/shelldemo x
[root@node-1 shelldemo]# vi ifdemo.sh
#!/bin/bash
num=100;
if [ $(num%2) == 0 ];then
    空格 echo "是偶数"; 空格
fi;

```

```

192.168.153.121 x
[root@node-1 shelldemo]# sh ifdemo.sh
是偶数
[root@node-1 shelldemo]#

```

案例2：条件判断

```
1  #!/bin/bash
2
3  num=10
4
5  if [  $\$(num\%2)$  == 0 ];then
6      echo "偶数";
7  else
8      echo "奇数";
9  fi
```

```
192.168.153.121 x
-rw-r--r--. 1 root root 75 5月  5 06:05 ifdemo.sh
[root@node-1 shelldemo]# vi ifdemo.sh
#!/bin/bash
num=101;
if [  $\$(num\%2)$  == 0 ];then
    echo "是偶数";
else
    echo "是奇数";
fi;
```

```
192.168.153.121 x
[root@node-1 shelldemo]# sh ifdemo.sh
是奇数
[root@node-1 shelldemo]#
[root@node-1 shelldemo]#
```

案例3：条件判断

```
1  #!/bin/bash
2
3  num1=10
4  num2=10
5
6  if [  $\$num1$  -gt  $\$num2$  ];then
7      echo " $\$num1$  >  $\$num2$ ";
8  elif [ $\$num1$  -lt  $\$num2$  ];then
9      echo " $\$num1$  <  $\$num2$ ";
10 else
```

```
11 echo "$num1 == $num2"
12 fi
```

```
192.168.153.121 x
[root@node-1 shelldemo]# vi ifdemo.sh
 1 #!/bin/bash
 2 num1=10;
 3 num2=20;
 4 if [ $num1 -gt $num2 ];then
 5     echo "$num1 > $num2";
 6 elif [ $num1 -lt $num2 ]; then
 7     echo "$num1 < $num2";
 8 else
 9     echo "$num1 == $num2";
10 fi;
```

```
192.168.153.121 x
[root@node-1 shelldemo]# sh ifdemo.sh
10 < 20
```

```
192.168.153.121 x
[root@node-1 shelldemo]# clear
[root@node-1 shelldemo]# vi ifdemo.sh
#!/bin/bash
num1=10;
num2=10;
if [ $num1 -gt $num2 ];then
    echo "$num1 > $num2";
elif [ $num1 -lt $num2 ]; then
    echo "$num1 < $num2";
else
    echo "$num1 == $num2";
fi;
```

```
192.168.153.121 x
[root@node-1 shelldemo]# clear
[root@node-1 shelldemo]# sh ifdemo.sh
10 == 10
```

7.2 for

循环流程控制：程序在执行时重复性的执行某行或某段代码。

- 不能出现死循环现象（在循环中添加条件用于在某个时刻结束循环）

一个简单的循环必须具备：

1. 循环初始值
2. 循环条件
3. 修改循环条件

```
1  for((循环初始值; 循环条件; 修改循环条件))
2  do
3      循环体代码（会重复执行的程序代码）
4  done
5
6  # 体育课跑5圈
7  for((count=0; count<5; count++ ))
8  do
9      循环体代码
10 done
```

方式一：用于数值方面的处理

```
1  for ((初始值; 限制值; 执行步长))
2  do
3      程序段
4  done
```

解释：

- 初始值：即循环初始值。例如：i=1
- 限制值：即循环条件。例如：i<=5
- 执行步长：即循环初始值修改。例如：i++

方式二的其它写法:

```
1 | for ((i = 0; i <= 5; i++)); do echo "$i"; done
```

案例1: 实现打印5次HelloWorld

```
1 |#!/bin/bash
2 |for ((i=0;i<5;i++))
3 |do
4 |    echo "$i - Hello World~"
5 |done
6 |
```

案例2: 求10以内数值的累加和

```
1 |#!/bin/bash
2 |
3 |# 定义变量
4 |count=0 # 存储累加计算的结果
5 |# 循环: 计算
6 |for((i=1;i<=10;i++))
7 |do
8 |    count=$((count+i));
9 |done
10 |# 输出计算后的结果
11 |echo "累加后的值: ${count}";
```

```
192.168.153.121 x
[root@node-1 shelldemo]# vi fordemo2.sh
#!/bin/bash
count=0;
for((i=1;i<=10;i++))
do
    count=$((count+i));
done
echo "累加后的值: ${count}";
```

```
192.168.153.121 x
[root@node-1 shelldemo]# sh fordemo2.sh
累加后的值: 55
```

方式二:

```
1 | for var in con1 con2 con3 ...
```

```
2 do
3     程序段..
4     echo $var
5 done
```

循环流程：

1. 第一次循环时，\$var的内容是con1
2. 第二次循环时，\$var的内容是con2
3. 第三次循环时，\$var的内容是con3

.....

也可以有其它写法：

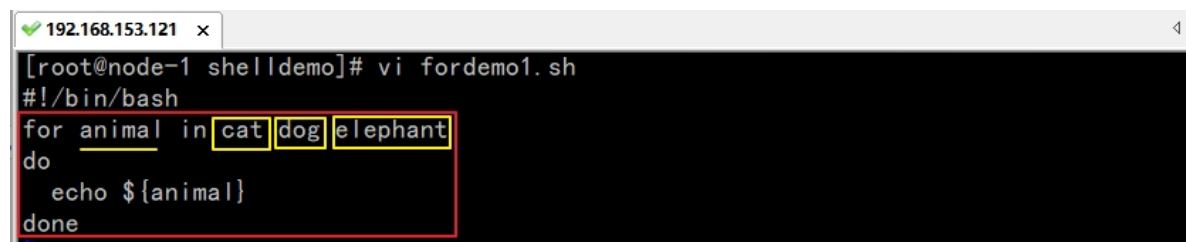
```
1 for N in 1 2 3; do echo $N; done
```

或

```
1 for N in {1..3}; do echo $N; done
```

案例1：

```
1 #!/bin/bash
2
3 for animal in cat dog elephant
4 do
5     echo ${animal}
6 done
```



```
192.168.153.121 x
[root@node-1 shelldemo]# vi fordemo1.sh
#!/bin/bash
for animal in cat dog elephant
do
    echo ${animal}
done
```

```
192.168.153.121 x
[root@node-1 shelldemo]# sh fordemo1.sh
cat
dog
elephant
[root@node-1 shelldemo]#
```

案例2：把指定目录下所有的文件名打印出来

```
1  #!/bin/bash
2
3  ###查询/root/shelldemo目录下所有的文件名，并赋值给filelist变量
4  filelist=$(ls /root/shelldemo);
5
6  ###循环遍历filelist，获取每一个文件名
7  for filename in $filelist
8  do
9      #输出文件名
10     echo $filename
11 done
```

```
192.168.153.121 x
[root@node-1 shelldemo]# vi fordemofile.sh
#!/bin/bash

###查询/root/shelldemo目录下所有的文件名，并赋值给filelist变量
filelist=$(ls /root/shelldemo);
###循环遍历filelist，获取每一个文件名
for filename in $filelist
do
    #输出文件名
    echo $filename
done
```

```
192.168.153.121 x
[root@node-1 shelldemo]# ls
demo1.sh demo2.sh fordemo1.sh fordemo2.sh fordemofile.sh hello.sh ifdemo.sh
[root@node-1 shelldemo]# sh fordemofile.sh
demo1.sh
demo2.sh
fordemo1.sh
fordemo2.sh
fordemofile.sh
hello.sh
ifdemo.sh
```

小结

- 循环流程：在程序执行过程中，针对某行或某段代码进行重复性的执行，至到循环条件不满足，才会结束重复执行操作
- 方式1：从指定的起始值开始循环，至到循环上限结束

```
1 for((循环初始值; 循环条件; 修改循环条件中的值))
2 do
3     程序代码
4 done
```

- 方式2：从一些数据集中，依次取出每一个数据进行操作，至到从数据集中取完所有数据

```
1 for 变量名 in 数据1 数据2 数据3 ....
2 do
3     程序代码
4 done
5 =====
6 for 变量名 in 数组
7 do
8     程序代码(循环体代码)
9 done
```

7.3 while

方式一：

```
1 while [ expression ]
2 do
3     command
4     ...
5     修改while中的循环条件
6 done
```

案例1：

```
1 #!/bin/bash
2
3 ###演示while循环的第一种方式
```

```
4 while [ "$y" != "yes" -a "$y" != "YES" ]
5 do
6     echo "请输入yes/YES停止循环: "
7     read y    ##接收键盘录入的值
8 done
9 echo "循环停止了!";
```

```
192.168.153.121 x
[root@node-1 shelldemo]# vi whiledemo1.sh
#!/bin/bash
###演示while循环的第一种方式
while [ "$y" != "yes" -a "$y" != "YES" ]
do
    echo "请输入yes/YES停止循环: "
    read y
done
echo "循环停止了!";
```

```
192.168.153.121 x
[root@node-1 shelldemo]# sh whiledemo1.sh
请输入yes/YES停止循环:
a
请输入yes/YES停止循环:
a
请输入yes/YES停止循环:
yes
循环停止了!
```

方式二:

```
1 i=1
2 while((i<=3))
3 do
4     echo $i
5     let i++    # (($i++))
6 done
```

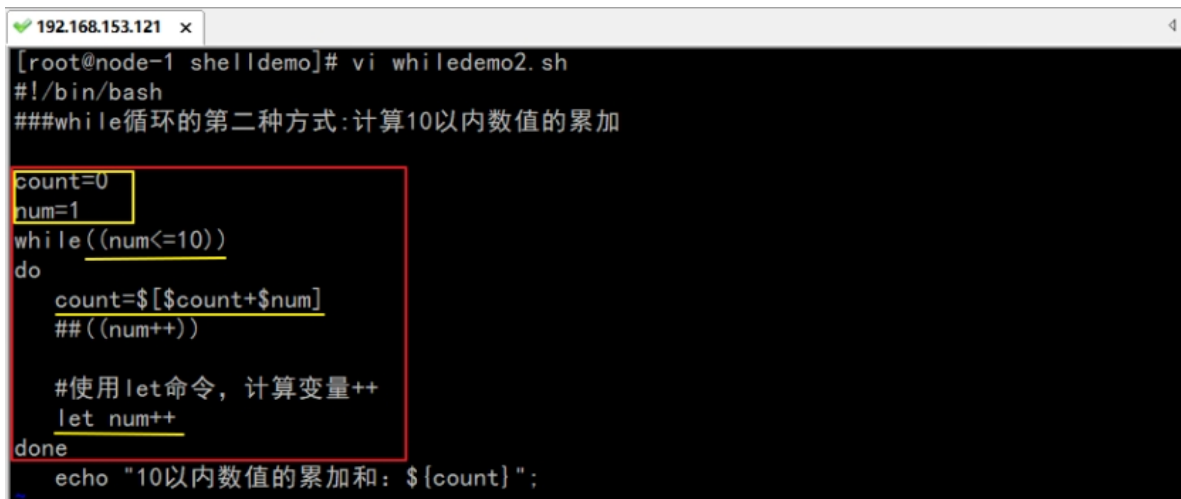
let 命令是 BASH 中用于计算的工具，用于执行一个或多个表达式，变量计算中不需要加上 \$ 来表示变量。

自加操作: `let no++`

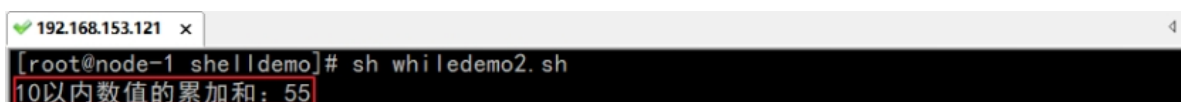
自减操作: `let no--`

案例2:

```
1  #!/bin/bash
2
3  ###while循环的第二种方式:计算10以内数值的累加
4  count=0 # 记录累加值结果
5  num=1 # 循环初始值
6  while((num<=10))
7  do
8      # 计算累加值的和
9      count=$((count+num))
10     ##((num++))
11     #使用let命令, 计算变量++
12     let num++ # 修改循环条件中的值
13 done
14 echo "10以内数值的累加和: ${count}";
```



```
192.168.153.121 x
[root@node-1 shelldemo]# vi whiledemo2.sh
#!/bin/bash
###while循环的第二种方式:计算10以内数值的累加
count=0
num=1
while((num<=10))
do
    count=$((count+num))
    ##((num++))
    #使用let命令, 计算变量++
    let num++
done
echo "10以内数值的累加和: ${count}";
```



```
192.168.153.121 x
[root@node-1 shelldemo]# sh whiledemo2.sh
10以内数值的累加和: 55
```

方式三: 无限循环(死循环)

```
1  while true
2  do
```

```
3 | command
4 | done
```

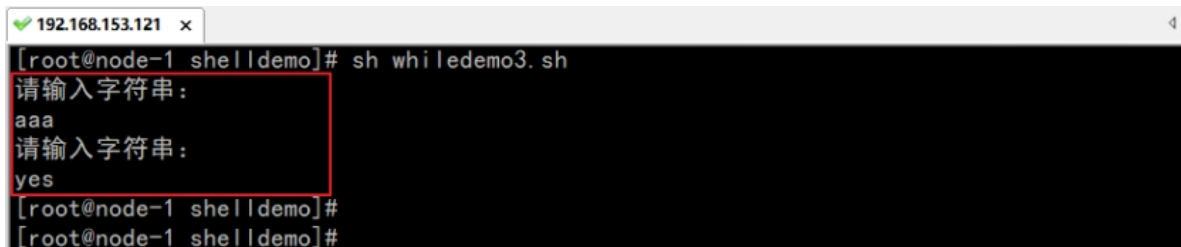
案例3:

```
1 | #!/bin/bash
2 | ###演示while死循环
3 | while true
4 | do
5 |     echo "请输入字符串: "
6 |     read y
7 |     if [ "$y" == "yes" ]; then
8 |         exit 0 ##退出
9 |     fi
10 | done
```



```
192.168.153.121 x 4
[root@node-1 shelldemo]# vi whiledemo3.sh
#!/bin/bash
###演示while死循环
while true
do
    echo "请输入字符串: "
    read y

    if [ "$y" == "yes" ]; then
        exit 0 ##退出
    fi
done
```



```
192.168.153.121 x 4
[root@node-1 shelldemo]# sh whiledemo3.sh
请输入字符串:
aaa
请输入字符串:
yes
[root@node-1 shelldemo]#
[root@node-1 shelldemo]#
```

7.4 case

格式:

```

1 case $变量名称 in
2
3     匹配模式1 )
4         程序段
5     ;; # 匹配模式1执行完毕
6
7     匹配模式2 )
8         程序段
9     ;;
10
11     * ) # 默认值, 没有匹配的模式
12         程序段
13     ;;
14
15 esac # 代表case语句结束

```

案例:

```

1 #!/bin/bash
2
3 ### 演示case的使用
4
5 case $1 in
6     "hello")
7         echo "Hello World!"
8     ;;
9
10    "test")
11        echo "testting..."
12    ;;
13
14    "")
15        echo "$0 没有参数"
16    ;;
17
18    *)
19        echo "默认"
20    ;;
21 esac

```



```
192.168.153.121 x
[root@node-1 shelldemo]# vi casedemo.sh
#!/bin/bash
### 演示case的使用

case $1 in
    "hello")
        echo "Hello World!"
        ;;
    "test")
        echo "testting..."
        ;;
    "")
        echo "$0 没有参数"
        ;;
    *)
        echo "默认"
        ;;
esac
```

```
192.168.153.121 x
[root@node-1 shelldemo]# sh casedemo.sh test
testting...
[root@node-1 shelldemo]# sh casedemo.sh hello
Hello World!
[root@node-1 shelldemo]# sh casedemo.sh 
casedemo.sh 没有参数
[root@node-1 shelldemo]# sh casedemo.sh other
默认
[root@node-1 shelldemo]#
```

8. 函数使用

函数：可以理解为是一个功能。例： `print`

- 好处：当有了解决某个问题的函数(功能)时，程序员就可以直接调用这个函数直接解决问题。

- 1 函数的划分：内置函数、自定义函数
- 2
- 3 内置函数：编程语言自身自带的函数。 例：shell语言中自带函数、awk语言自带函数(print)
- 4
- 5 自定义函数： 程序员自己编写的函数。
- 6 | -- 函数的定义（如何编写一个函数）
- 7 | -- 函数的使用（如何调用编写好的函数）

我们要学习如何制作：函数、调用函数

在 Shell 中所有函数在使用前必须定义。这意味着必须将函数放在脚本开始部分，直至 shell 解释器首次发现它时，才可以使用。调用函数仅使用其函数名即可。

- 先把函数制作好，才能正确调用

记住：Shell 中的函数书写在前面（书写在调用函数的代码之前）

```
1 function 函数名字 ()
2 {
3     程序段;
4     [return int;]
5 }
```

- 1、可以带 `function fun()` 定义，也可以直接 `fun()` 定义, 不带任何参数。
- 2、参数返回，可以显示加 `return`，如果不加，将以最后一条命令运行结果，作为返回值。

8.1 函数的简单使用

案例：函数的使用

```
1 #!/bin/bash
2
3 ### 演示函数的简单使用
4 ##注意：函数书写在shell脚本的开始位置
5 ##定义函数，函数名为：print
6 function print()
7 {
8     echo "hello"
9     echo "你好"
10 }
11
12 ##调用print函数
13 print
```

```
192.168.153.121 x
[root@node-1 shelldemo]# vi functiondemo1.sh
#!/bin/bash
### 演示函数的简单使用

##注意：函数书写在shell脚本的开始位置
##定义函数，函数名为：print
print()
{
    echo "hello"
    echo "你好"
}

##调用print函数
print
```

```
192.168.153.121 x
[root@node-1 shelldemo]# sh functiondemo1.sh
hello
你好
[root@node-1 shelldemo]#
```

8.2 函数的参数

在 Shell 中，调用函数时可以向其传递参数。在函数体内部，通过 `$n` 的形式来获取参数的值，例如，`$1` 表示第一个参数，`$2` 表示第二个参数...

注意，当 $n \geq 10$ 时，需要使用 `${n}` 来获取参数。

案例：带参数的函数

```
1  #!/bin/bash
2
3  ### 演示带参数的函数
4  ##定义函数
5  funWithParam()
6  {
7      echo "第一个参数为 $1"
8      echo "第二个参数为 $2"
9      echo "第十个参数为 $10"
10     echo "第十个参数为 ${10}"
```

```

11 echo "第十一个参数为 ${11}"
12 echo "参数总数有 $# 个"
13 echo "作为一个字符串输出所有参数 $*"
14 }
15 ##调用函数，并传递参数
16 funWithParam 1 2 3 4 5 6 7 8 9 34 73

```

```

192.168.153.121 x
[root@node-1 shelldemo]# vi functiondemo2.sh
#!/bin/bash
### 演示带参数的函数
##定义函数
funWithParam()
{
echo "第一个参数为 $1"
echo "第二个参数为 $2"
echo "第十个参数为 $10"
echo "第十个参数为 ${10}"
echo "第十一个参数为 ${11}"
echo "参数总数有 $# 个"
echo "作为一个字符串输出所有参数 $*"
}
##调用函数，并传递参数
funWithParam 1 2 3 4 5 6 7 8 9 34 73

```

```

192.168.153.121 x
[root@node-1 shelldemo]# sh functiondemo2.sh
第一个参数为 1
第二个参数为 2
第十个参数为 10
第十个参数为 34
第十一个参数为 73
参数总数有 11 个
作为一个字符串输出所有参数 1 2 3 4 5 6 7 8 9 34 73
[root@node-1 shelldemo]#

```

```

1 ./hello.sh 1 5 #调用shell程序时传递两个数据：1和5
2
3 #!/bin/bash
4 num1=$(( $1++ ))
5 num2=$(( $2++ ))
6
7 # 定义函数：接收两个数据
8 function sum(){
9     # 计算两个数字之和
10    sum=$(( $1+$2 ))
11    echo ${sum}
12 }

```

```
13
14 # 调用函数
15 sum $num1 $num2
```

8.3 函数的返回值

函数的返回值：函数内部在处理完所有问题后，需要有一个结果返回给调用者。

```
1 function 函数名(){
2     .....
3     函数体中的代码
4     .....
5
6     return 数据  ## 返回的数据只能有一个值
7 }
```

案例：有返回值的函数

```
1 #!/bin/bash
2 ###演示带返回值的参数
3 function getMax()
4 {
5     if [ $1 -lt $2 ]; then
6         return $2 #使用return来返回一个数据值 （return后面的返回值
7         只能书写一个）
8     else
9         return $1;
10    fi
11 }
12
13 echo "Shell程序中传递的两个参数是：$1 , $2"
14
15
16 ##调用函数
17 getMax $1 $2
18
19 echo "最大值： : $?" # $?表示返回值
```

```
192.168.153.121 x
[root@node-1 shelldemo]# vi functiondemo3.sh
#!/bin/bash
###演示带返回值的参数
function getMax()
{
    if [ $1 -lt $2 ]; then
        return $2
    else
        return $1;
    fi
}

echo "Shell程序中传递的两个参数是: $1, $2"

##调用函数
getMax $1 $2

echo "最大值: $?"  $? 表示函数的返回值
```

```
192.168.153.121 x
[root@node-1 shelldemo]# sh functiondemo3.sh 100 20
Shell程序中传递的两个参数是: 100 , 20
最大值: 100
[root@node-1 shelldemo]# sh functiondemo3.sh 10 20
Shell程序中传递的两个参数是: 10 , 20
最大值: 20
[root@node-1 shelldemo]#
[root@node-1 shelldemo]#
```

小结：

函数的定义：

```
1 | function 函数名()
2 | {
3 |     .....
4 | }
```

函数的调用： 函数名

函数中的参数传递：

- 在函数中接收传递的参数可以使用： `$n` 例： `$1 $7 ${10}`
- 调用函数时传递参数： 函数名 参数1 参数2

函数调用后接收返回值：

- 在函数中使用 `return 数据值` 的方式把数据返回出去

- 函数的返回值默认是存储在： `$?`
- 可以直接使用 `$?` 操作返回值

9. 数组

数组可以理解为是一个用来存放多个值的容器。

数组中存储的数据，通常可以称为：元素。

当数组中存储了多个元素后，就会给每一个元素添加一个编号(索引)，从0开始。

9.1 定义数组

Bash Shell 只支持一维数组（不支持多维数组），初始化时不需要定义数组大小。

Shell 数组用括号来表示，元素用"空格"符号分割开，语法格式如下：

```
1 | array_name=(value1 value2 value3 ... valueN)
```

示例：

```
1 | #!/bin/bash
2 | my_array=(A B "C" D)
```

也可以使用索引来定义数组：

```
1 | # 数组名[索引]=元素值
2 | array_name[0]=value0
3 | array_name[1]=value1
```

9.2 读取数组

读取数组元素值的一般格式是： `${array_name[index]}`

- `index` 是指数组的索引，从0开始

9.2.1 示例

```
1 #!/bin/bash
2 my_array=(A B "C" D)
3 echo "第一个元素为: ${my_array[0]}"
4 echo "第二个元素为: ${my_array[1]}"
5 echo "第三个元素为: ${my_array[2]}"
6 echo "第四个元素为: ${my_array[3]}"
```

- 执行脚本输出结果：

```
1 第一个元素为: A
2 第二个元素为: B
3 第三个元素为: C
4 第四个元素为: D
```

9.2.2 获取数组中的所有元素

使用 `@` 或 `*` 可以获取数组中的所有元素，例如： `${array_name[*]}`、`${array_name[@]}`

示例：

```
1 #!/bin/bash
2 my_array[0]=A
3 my_array[1]=B
4 my_array[2]=C
5 my_array[3]=D
6 echo "数组的元素为: ${my_array[*]}"
7 echo "数组的元素为: ${my_array[@]}"
```

- 执行脚本，输出结果如下所示：

```
1 数组的元素为: A B C D
2 数组的元素为: A B C D
```


9.2.3 获取数组的长度

获取数组长度的方法与获取字符串长度的方法相同，例如：`${#array_name[*]}`、`${#array_name[@]}`

示例：

```
1 #!/bin/bash
2 my_array[0]=A
3 my_array[1]=B
4 my_array[2]=C
5 my_array[3]=D
6 echo "数组元素个数为: ${#my_array[*]}"
7 echo "数组元素个数为: ${#my_array[@]}"
```

- 执行脚本，输出结果如下所示：

```
1 数组元素个数为: 4
2 数组元素个数为: 4
```

9.3 遍历数组

遍历：把数组中存储的元素依次取出

- 要使用：循环

回顾for循环的知识点：

```
1 # 第一种方式
2 for((i=0;i<上限值;i++))
3 do
4     程序....
5 done
6
7 # 第二种方式
8 for 变量名 in 数据1 数据2 数据3
9 do
```

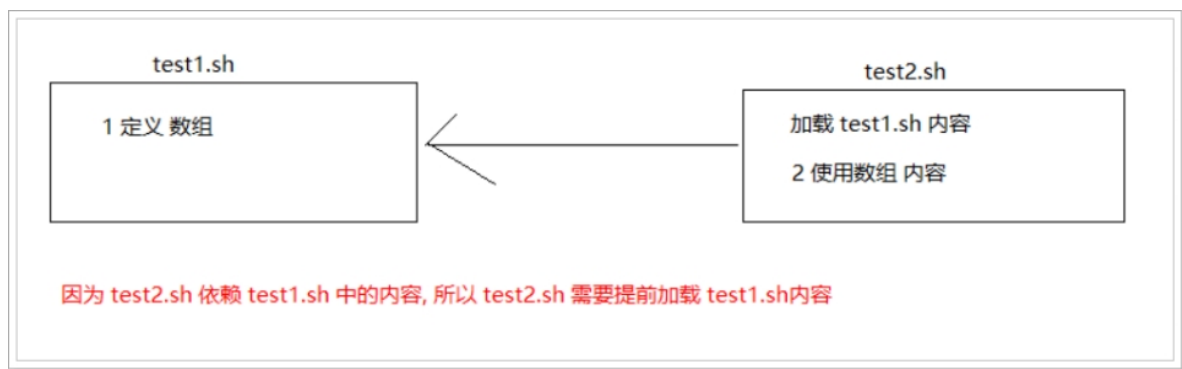
```
10 程序.....
11 done
```

方式1: 遍历数组

```
1 #!/bin/bash
2 # 定义数组
3 my_arr=(AA BB CC)
4 # 利用for循环 遍历数组
5 for var in ${my_arr[*]}
6 do
7     echo $var
8 done
```

方式2: 遍历数组

```
1 #!/bin/bash
2 my_arr=(AA BB CC) #定义数组
3 my_arr_num=${#my_arr[*]} #获取数组的长度
4 for((i=0;i<my_arr_num;i++));
5 do
6     echo ${my_arr[$i]}
7 done
```



10.1 简介

在一个Shell程序中可以指定包含外部的其他Shell脚本程序。这样可以很方便的封装一些公用的代码作为一个独立的文件。

Shell 文件包含的语法格式如下：

```
1 | . 空格 filename    # 注意点号"."和文件名中间有一空格
```

或者

```
1 | source filename
```

10.2 示例

定义两个shell文件分别为： test1.sh、test2.sh

1. 在test1中定义一个变量 `arr=(linux hadoop shell)`
2. 在test2中对arr进行循环打印输出

第一步: vim test1.sh

```
1 | #!/bin/bash
2 | my_arr=(AA BB CC)
```

第二步: vim test2.sh

```
1 | #!/bin/bash
2 | source /export/shelldemo/test1.sh    # 加载test1.sh 的文件内容
```

```
3 for var in ${my_arr[*]}
4 do
5     echo $var
6 done
```

好处：

1. 数据源和业务处理分离
2. 复用代码扩展性更强

11. Shell脚本案例

11.1 猜数字游戏

11.1.1 规则

游戏规则为：程序内置一个1到100 之间的数字作为猜测的结果，由用户猜测此数字。用户每猜测一次，由系统提示猜测结果：大了、小了或者猜对了；直到用户猜对结果，则提示游戏结束。

11.1.2 代码实现

guess_number.sh

```
1 #!/bin/bash
2 # 生成100以内的随机数 提示用户猜测 猜对为止
3 # random 系统自带，值为0-32767任意数
4 # 随机数1-100
5 num=$((RANDOM%100+1))
6
7 while true
8 do
9     # read 接收用户录入的数字
10    read -p "计算机生成了一个 1-100 的随机数,你猜: " cai
```

```
11
12     # if判断
13     if [ $cai -eq $num ]
14     then
15         echo "恭喜,猜对了"
16         exit #退出循环
17     elif [ $cai -gt $num ]
18     then
19         echo "不巧,猜大了"
20     else
21         echo "不巧,猜小了"
22     fi
23 done
```

